What Happens When You Turn a Card Game into an NP-Complete Problem? (A Personal Exploration)

Overview

From an engineering perspective, I designed and implemented a turn-based PvE card game using the Unity engine. The game features simple rules yet requires both luck and strategic decision-making. The computer opponent's strategy is powered by a locally deployed server running a trained ensemble AI model.

Theoretically, I abstracted the game mechanics into a subsequence selection and score maximization problem governed by specific rules. Through in-depth analysis, I discovered that the actual decision space grows combinatorially with the number of moves, leading to an explosive increase in complexity. To understand its computational complexity, I examined a generalized version of this card game. Suspecting the abstracted problem might be NP-Hard, I formally reduced it to the Subset Sum problem, thereby proving that the problem is not only NP-Hard but also NP-Complete. Consequently, exact solutions become computationally infeasible even for moderately sized instances.

To address this challenge, I explored a general-purpose solution approach based on a lightweight surrogate model that integrates a Transformer architecture with a genetic algorithm. The core idea is to leverage an exact exhaustive-search algorithm to generate high-quality training samples within a computationally tractable problem scale, then train a computationally inexpensive AI model to mimic the optimal decision-making behavior observed in those samples.

This document is not a formal academic paper, nor does it claim to present rigorously validated academic contributions. Rather, it functions as a high-fidelity project report or a personal portfolio piece—demonstrating both theoretical understanding and engineering capabilities within a carefully designed sandbox environment. The work achieves a meaningful depth of exploration and incorporates several innovative elements, aiming to showcase my proficiency and readiness in applying advanced computational techniques to complex real-world-inspired problems.

Project Highlights

- **Abstraction and Implementation of Problem Modeling**
- **Rigorous Algorithmic Analysis**
- **Proof of NP-Completeness for a Custom-Designed Problem, with Supporting Discussion**

  > *Note: The core proof appears in the main text; extended discussion is provided in the appendix.*

- **Comprehensive AI Model Evaluation Framework**
- **Design, Implementation, and Analysis of a Surrogate Model, including Discussions on Generalization and Robustness**
- **Batch Processing and Multithreading Optimizations for the Ensemble Model**
- **Custom Development and Extension of a Relatively Complex Game Framework**

- **End-to-End System Design and Implementation**
- **Discussion of a Tree-Based Representation Well-Suited for Expressing Combinatorial Problems (Appendix)**

> Note: The actual code implementation is available at https://github.com/tcita/python_server.git

Document Structure

This document is divided into four main sections:

1. Game Rules and Algorithmic Analysis

This section provides a detailed description of the game rules and their programmatic implementation, including:

- **Game Rules**
- **Algorithmic Analysis**
- **Proof of NP-Completeness**
- **Discussion on Algorithmic Optimizations and the Proposal of an AI Surrogate Model**

2. AI Technical Details

This section delves into the inner workings of the AI system used in the game and its core implementation, covering:

- **AI Model Implementation**
- **Model Training**
- **Ablation Studies on Data Preprocessing**
- **Model Evaluation**
- **Ensemble Model Evaluation and Optimization**
- **Discussion on Modeling Paradigm Conflicts and Inductive Biases in the Transformer Architecture**

3. Server Deployment and Game Architecture

This section presents the Unity client architecture and live gameplay demonstration, including:

- **AI Model Server Setup**
- **Client-Side Game Architecture**
- **Live Gameplay Demonstration**

4. Appendix, Author’s Self-Reflection (Q&A), and Other Supplementary Materials

Part I: Game Rules and Algorithmic Analysis

Game Rules and Gameplay Example

The game rules are somewhat similar to "Tile Matching" (card game). Players score points by pairing cards to eliminate them. When it's the computer's turn, the current game state is captured, and the dynamic card layout data is sent to a local AI server. The AI returns its predicted optimal move, which is then automatically

executed and displayed to the user on the client side. After three rounds, the player with the higher total score wins.

Initial Setup

1. The system creates a list **P** containing 52 numbers, where each integer from 1 to 13 appears exactly four times (13 × 4 = 52 elements in total).
2. **Six distinct elements** are randomly drawn from list **P** to form list **A**.
    - **Example:** Suppose we obtain **A = {1, 2, 3, 7, 4, 6}**.
    - **(Note)** In this specific study, the number of elements in **A** is fixed at 6.
3. Then, **three elements (allowing duplicates)** are randomly drawn from the remaining elements of **P** to form list **B**.
    - **Example:** Suppose we obtain **B = {4, 3, 9}**.
    - **(Note)** In this specific study, the number of elements in **B** is fixed at 3.

Insertion Positions

- Valid insertion positions include all gaps between elements of **A** as well as both ends.
- For the example **A = {1, 2, 3, 7, 4, 6}**, the insertion positions (numbered) are: ① 1 ② 2 ③ 3 ④ 7 ⑤ 4 ⑥ 6 ⑦
- There are a total of 6 + 1 = 7 possible insertion positions.

Start of a Turn

- On their turn, a player selects one card from **B** and inserts it into one of the valid insertion positions in **A**.
- Once selected and inserted, the card is removed from **B** and added to **A**.

Example Turn Sequence

First Move

1. The player chooses to take the card 3 from **B** and insert it into position ① of **A** (i.e., before **A[0]**).
2. **A** and **B** are updated:
    - **A** becomes **{3, 1, 2, 3, 7, 4, 6}**
    - **B** becomes **{4, 9}**

Matching Rule

- After insertion, examine the newly inserted card (3). Search **A** for any other element with the same numerical value.
- **Example:** The newly inserted card is 3. In **{3, 1, 2, 3, 7, 4, 6}**, **A[3]** also equals 3, forming a match with the newly inserted 3.
- **(Note)** At most one matching element exists in **A** for any newly inserted card. This is guaranteed because **A** initially contains only unique values.

Scoring and Removal (First Move)

1. Upon finding a match, define a sequence starting from the newly inserted card up to (and including) the matched element.

    - **Example:** The new 3 is at **A[0]**, and the matched 3 is at **A[3]**. The sequence is **A[0]** through **A[3]**: **{3, 1, 2, 3}**.
2. This matched sequence is removed from **A**.
3. Score is calculated as the sum of all values in the sequence.
    - **Example:** $Score_1 = 3 + 1 + 2 + 3 = 10$.
4. **A** is dynamically updated:
    - After removing **{3, 1, 2, 3}**, **A** becomes **{7, 4, 6}**.
    - **B** remains unchanged as **{4, 9}**.
5. Insertion positions are updated accordingly: for **A = {7, 4, 6}**, valid positions become ① 7 ② 4 ③ 6 ④.

Second Move

1. The player chooses to take the card 9 from **B** and insert it into position ④ of **A**.
2. **A** and **B** are updated:
    - **A** becomes **{7, 4, 6, 9}**.
    - **B** becomes **{4}**.
3. Check for matches: the newly inserted card is 9. In **{7, 4, 6, 9}**, no other element equals 9.
4. No match occurs, so $Score_2 = 0$.
5. **A** and **B** remain in their current states.
6. Insertion positions become ① 7 ② 4 ③ 6 ④ 9 ⑤.

Third Move

1. The player takes the last remaining card 4 from **B** and inserts it into position ⑤ of **A**.
2. **A** and **B** are updated:
    - **A** becomes **{7, 4, 6, 9, 4}**.
    - **B** becomes **{}** (empty).
3. Check for matches: the newly inserted card is 4. In **{7, 4, 6, 9, 4}**, **A[1]** also equals 4, matching the newly inserted **A[4]**.
4. The matched sequence is from **A[1]** to **A[4]**: **{4, 6, 9, 4}**.
5. Score is recorded: $Score_3 = 4 + 6 + 9 + 4 = 23$.
6. After removing **{4, 6, 9, 4}**, **A** becomes **{7}**.
7. **B** is already empty (**{}**).

End of Turn

- All players (in this example, just the human player) have played three cards, and all elements from **B** have been inserted into **A**.
- The turn ends.

Total Score

- The total score is the sum of all scores obtained during the turn.
- **Example:** Total Score = $Score_1 + Score_2 + Score_3 = 10 + 0 + 23 = 33$.

Exhaustive Recursive Algorithm Analysis

To compute the optimal strategy for a given pair of lists **A** and **B**, we first clarify the following key concepts:

Given A and B:

- **A** is an integer list of length *m*, with all elements distinct, each in the range 1 to *K*.
- **B** is an integer list of length *n*, with elements allowed to repeat, each also in the range 1 to *K*.

Strategy:
In this context, a *strategy* specifies **both**:

1. The **order** in which elements from **B** are inserted, and
2. The **insertion positions** in **A** where each selected element is placed.

Thus, a complete strategy fully determines the sequence of moves: for each element taken from **B**, we decide *when* (in which turn) to play it and *where* (at which valid position) to insert it into the current **A**.

Matching:
Immediately after selecting a card from **B** and inserting it into a specific position in **A**, we perform the following check:
① Whether the value of the newly inserted card matches the value of **any pre-existing card** already present in **A**.
② If such a matching card exists in **A**, we say a **"match"** has occurred.

Match Sequence and ***Removal***:
When a match occurs, a **"match sequence"** is identified. This sequence includes:
① The newly inserted card,
② The pre-existing card in **A** with the same value, and
③ All cards located **between** these two cards in the current list **A**.

This entire contiguous subsequence is then **removed** from **A**.

Score:
The score for this move equals the **sum of all values** in the removed match sequence.

No Match and Zero Score:
If the value of the newly inserted card does **not** appear anywhere else in the current **A**, then **no match** occurs. In this case, the match sequence is empty, nothing is removed from **A**, and the score for this move is **0**.

Total Score:
Applying a given strategy means sequentially inserting all elements of **B** into **A** according to the specified order and positions. Each insertion triggers a match check, yields a score (possibly zero), and potentially modifies **A**. The **total score** is the sum of the scores from all *n* insertions (in our specific problem, $n = 3$).

Best Strategy:
The **best strategy** is the one that **maximizes the total score** over all possible strategies.

First, we perform a generalized mathematical analysis of the problem:

Verification Complexity Analysis:

This computes the time complexity required to determine the outcome—according to the game rules—given arrays A and B and a specified insertion position(strategy).

For each element in **B**, the algorithm attempts to insert it into every possible position in the current **A**, then recursively processes the remaining elements. Suppose $|B| = n$ and $|A| = m$. In the first step, there are n choices for which element to insert, and for each such choice, there are $m + 1$ possible insertion positions. Thus, there are $n \cdot (m + 1)$ possible paths at the first level.

After inserting an element at an arbitrary position, let p denote the probability that a match occurs.

Expected Length of the Removed Subsequence Upon a Match

We now analyze the expected length of the subsequence removed when a match occurs.

Consider the following simplified scenario:
We have a sequence **A** of m distinct integers. A value $x \in A$ already exists in **A**. Now we insert another copy of x into a uniformly random position among the $m + 1$ possible insertion slots, forming a new sequence **A'**. This creates a contiguous subsequence $[x \dots x]$, which is then removed. What is the expected length of this removed subsequence?

This can be further reduced to a purely combinatorial problem:

> From the integers $1, 2, \dots, m + 1$, select two distinct numbers uniformly at random. What is the expected number of integers between them, inclusive?

Let the two selected positions be i and j with $i < j$. The distance (inclusive length) is $d = j - i + 1$, where $2 \le d \le m + 1$.

The expected value is:

$$\mathbb{E}[d] = \frac{\sum_{d=2}^{m+1} d \cdot (m - d + 2)}{\binom{m+1}{2}} = \frac{(m+2)\sum_{d=2}^{m+1} d - \sum_{d=2}^{m+1} d^2}{\binom{m+1}{2}}$$

Applying the formulas for the sum of an arithmetic series and the sum of squares, and simplifying, we obtain:

$$\mathbb{E}[d] = \frac{m+5}{3}$$

Therefore, **the expected length of the removed subsequence upon a successful match is $\frac{|A|+5}{3}$**.

Consequently, if the current length of **A** is $|A|$, then after a match, the expected new length $|A'|$ is:

$$\mathbb{E}[|A'|] = |A| + 1 - \frac{|A|+5}{3} = \frac{2|A|-2}{3}$$

Expected Matching Probability Over Time

Now consider the matching probability dynamics.

We have a sequence **A** of length m with all distinct elements. We insert an element b (from **B**, which may contain duplicates) into a random position in **A**. With probability p, a match occurs; otherwise (with probability $1 - p$), nothing is removed.

Let:

- $E[K]$: expected number of elements in **A** that can match any element in the remaining **B**.
- $E[K']$: expected number of such matchable elements in **A** after a successful match.

Due to the linearity of expectation and the earlier result on sequence length reduction, after a match:

$E[K'] \approx \frac{2E[K]-2}{3}$

Meanwhile, the size of **B** decreases by 1.

The matching probability at any step is approximately:

$p = \frac{\text{number of matchable elements in } A}{\text{size of remaining } B}$

Thus, after a successful match:

- Numerator (matchable count) shrinks roughly by a factor of $2/3$,
- Denominator (size of **B**) decreases by 1.

Assuming large values ($x = E[K] \gg 1$, $y = |B| \gg 1$), we observe:

$\frac{x}{y} > \frac{(2x-2)/3}{y-1} \approx \frac{2x}{3(y-1)} \approx \frac{2}{3} \cdot \frac{x}{y}$

Hence, **the matching probability p decreases after a successful match**.

Conversely, if no match occurs:

- **A** grows by 1 (since the new element is added without removal),
- The new element may itself become a future match target, so the number of matchable elements **increases or stays the same**,
- **B** still shrinks by 1.

Therefore, **p increases significantly after a failed match**.

This leads to a key insight:

> The system exhibits a **negative feedback loop**:
>
> - If a match occurs → p decreases;
> - If no match occurs → p increases.

This behavior aligns with intuition and stabilizes the system.

State Evolution Model

Define M_t as the length of **A** at the beginning of step t. The system evolves as:

- With probability p_t (match succeeds): $M_{t+1} = \frac{2M_t-2}{3}$
- With probability $1 - p_t$ (no match): $M_{t+1} = M_t + 1$

Here, p_t depends on the overlap between the current **A** and the remaining **B**.

> Due to the negative feedback described above, the system self-regulates:

> - Elimination reduces M → lowers future p_t;
> - No elimination increases M → raises future p_t.

Thus, M_t neither explodes nor collapses to zero, but fluctuates around a **dynamic equilibrium**.

Steady-State Analysis

Assume the system reaches a steady state where the expected length remains constant: $\mathbb{E}[M_t] \approx M$ for all t. Approximate $p_t \approx p$ as a constant depending only on M and the current size of **B** (denoted y).

Then:

$$M = p \cdot \frac{2M-2}{3} + (1-p)(M+1)$$

Solving for M:

$$M = \frac{3}{p} - 5$$

Estimating the Equilibrium Length

To estimate p and locate the balance point, we adopt a **mean-field approximation**:

- Although **B** is a fixed input sequence, **A** evolves dynamically.
- Both share the same alphabet (values 1 to K).
- In expectation, the number of matchable elements between **A** and the remaining **B** is proportional to M.

We make two key assumptions:

1. **Mean-field approximation**: In the large-system limit, the instantaneous matching probability can be approximated by its expected value, and this probability is proportional to the current length M of sequence A.
2. **Quasi-static approximation**: Since $y = |B|$ changes slowly (decreases by 1 per step), the system quickly equilibrates at each fixed y.

Thus:

$$p_t \approx c \cdot \frac{M_t}{y} \quad (\text{for some constant } c)$$

In equilibrium, $M \approx 3/p - 5 \approx 3/p$ (ignoring the constant for scaling analysis). Substituting $p \approx M/y$:

$$M \approx \frac{3}{p} \approx \frac{3y}{M} \quad \Rightarrow \quad M^2 \approx 3y$$

Therefore, for most of the process (when $y = \Theta(n)$):

$$M_t = \Theta(\sqrt{n})$$

This implies that the typical length of the dynamic sequence **A** scales with the square root of the initial size of **B**.

Since $M_t = \Theta(\sqrt{n})$ holds in typical cases, the total time complexity is:
$T = \sum_{t=1}^{n} O(M_t) \approx \sum_{t=1}^{n} O(\sqrt{n}) = O(n \cdot \sqrt{n}) = O(n^{3/2})$.

More precisely, if we treat $M_t \approx \sqrt{n - t}$ as a continuous function, then:
$T \approx \int_0^n \sqrt{t}, dt = \frac{2}{3} n^{3/2} = \Theta(n^{3/2})$.

This model is primarily intended for analyzing the macroscopic statistical behavior of the system when $M \gg 1$. We assume that the probability of the system being completely emptied [i.e., $M = 0$ or $M = 1$] during execution is negligible. The validity of this assumption stems from the fact that the equilibrium point established by the negative feedback mechanism, $M_{\text{eq}} \approx \sqrt{3y/c}$, decreases slowly as y diminishes, yet remains far from zero as long as y is still relatively large. Consequently, the system spends the vast majority of its time in states where M is sufficiently large.

Our analysis characterizes the system's behavior in this steady-state regime and does not precisely capture the dynamics near absorbing or reflecting boundaries at very small values of M—for example, if all elements of A are completely eliminated after being matched by a subset of B, then subsequent insertions of B would cause elements from B to match among themselves.

Research on a Specific Instance of the Original Problem

In order to obtain concrete data for training AI and implementing a Unity game, we investigate a specific instance of the original problem where array A has length 6, array B has length 3, and K = 13.

Problem Modeling

First, we develop the `simulate_insertion_tool()` function capable of handling the insertion process and scoring. Since array A dynamically changes due to insertions and subsequent matches, when this function is called, the input length of A is no longer the initial length of 6, but rather ranges from 0 to 8 (as A can only be in the range 0–8 when `insertion` might be called—A cannot be negative, and when A reaches length 9, B becomes empty and no further elements can be inserted from B). After insertion and any resulting matches, A's length will be in the range 0–9.

We define the function `simulate_insertion_tool(A, b, pos)` (The implementation in code_path), which inserts element `b` into list `A` at the specified position `pos` and handles any matching that may occur.

The function returns: `[score, length_removed_due_to_matching, new_length_of_A, match_occurred (0 or 1), new_A]`.

One key challenge in this problem is designing an appropriate data structure to represent insertion strategies.

The strategy structure I use is a 3×2 two-dimensional matrix (or list), which we refer to as `move`. Each row of this matrix represents one insertion operation, with two columns:

- The first column indicates the index of the element in array B;
- The second column specifies the position in the current array A where this element should be inserted.

Importantly, the "insertion position" is always relative to the most up-to-date state of A, not the initial A.

For example:

```
B = [1, 2, 3]
move = [[1, 6], [0, 3], [2, 0]]
```

This strategy means:

- Step 1: Insert B[1] (i.e., element 2) into the current A at index position 6;
- Step 2: Insert B[0] (i.e., element 1) into the updated A at index position 3;
- Step 3: Insert B[2] (i.e., element 3) into the A (updated again) at index position 0.

Each step operates on the current version of A, and every insertion of B[i] causes A to be updated due to both the insertion itself and any subsequent matching.

Thus, we can design a recursive algorithm:

`Recursive_StrategyAndScore(A, B, original_indices=None, move_sequence=None, used_indices=None)` (implementation available at code_path)

which returns: **the theoretical maximum score** and **the corresponding sequence of moves that achieves this maximum score**.

Estimating the Scale of Exhaustive Search for Optimal Decision Paths under the Specific Instance

We compute the total number of possible final static configurations when inserting all elements of B into A, assuming no specific values for A and B are given and **no matching or elimination occurs at all**. Note that this count represents only the number of distinct *final static arrangements*, not the full decision process. Through static modeling, we estimate the theoretical upper bound on the number of possible final hand combinations.

Consider the scenario where we insert all elements of B (an array of length 3) into A (an array of length 6), ignoring the actual numeric values (i.e., treating cards only as belonging to group A or group B). This becomes a multiset permutation problem: how many distinct length-9 arrays can be formed by arranging the multiset {A, A, A, A, A, A, B, B, B}?

The answer is: $\binom{9}{6} \cdot \binom{3}{3} = \binom{9}{6} = \binom{9}{3}$

Now, we assign actual numeric values (from 1 to 13) to these placeholders:

- Each of the 6 A-cards must have **distinct** values from 1 to 13, so the number of ways to assign values to A is the number of permutations: $P(13, 6) = \frac{13!}{(13-6)!}$.
- Each of the 3 B-cards can take **any** value from 1 to 13, with repetitions allowed, giving 13^3 possible assignments.

Therefore, the total number of possible final configurations (under the no-matching assumption) is:
$\binom{9}{3} \cdot \frac{13!}{(13-6)!} \cdot 13^3 = 228{,}012{,}744{,}960$

Assuming each sample requires approximately 10 bytes of storage, the total data size would exceed **2,000 GB**—an enormous amount.

> **Important Caveats**:
> This is only a rough estimate. As noted earlier, in the actual game, each insertion may trigger matching and removal, dynamically altering the length and content of A. Consequently, the state of A changes after each insertion, meaning that the order of insertions matters significantly.
>
> For example, inserting b_1 at position 1 followed by b_2 at position 2 can yield a different outcome than inserting b_2 at position 2 first and then b_1 at position 1—due to intermediate matching effects. Thus, these are two distinct decision paths.
>
> Moreover, our calculation assumes **no matches occur**, which also implies that some insertion sequences might be invalid in practice (e.g., attempting to insert at a position beyond the current length of A). Matching not only changes A's length but also introduces complex state dependencies. It remains unclear whether matching ultimately increases or decreases the total number of feasible decision paths—but it certainly makes the true search space highly non-trivial and far more intricate than this static upper bound suggests.

A Better Estimate of Data Scale via Monte Carlo Simulation

To estimate the real impact of matching, we employed probabilistic statistical analysis—specifically, Monte Carlo simulation—for the specific case where `length(A) = 6` and `length(B) = 3`.

In each trial:

- Random arrays A (length 6) and B (length 3) are generated.
- One card is randomly selected from B (without replacement) and inserted at a random valid position in A.
- The `simulate_insertion_tool` function is called to perform the insertion and handle any resulting matches.
- This process is repeated **twice**: the second insertion operates on the updated A and reduced B from the first step.
- We record the third return value of `simulate_insertion_tool`—the new length of A after matching.

This entire procedure was repeated **20 million times** (`total_times = 20,000,000`) for statistical robustness. Below is the Python-like pseudocode used:

```
{
    A, B = get_A_and_B_random()

    card = random.choice(B)
    pos = random.randint(0, 6)
    _, _, new_length1, _, newA = simulate_insertion_tool(A, card, pos)
    all_lengths1.append(new_length1)

    B.remove(card)
    card = random.choice(B)
    pos = random.randint(0, len(newA))
    _, _, new_length2, _, newA = simulate_insertion_tool(newA, card, pos)
```

```
    all_lengths2.append(new_length2)
}
```

Simulation Results (20M trials):

Distribution of A's length after the first insertion (`A1`):

- `length(A1) = 0`: 559,058 times (2.80%)
- `length(A1) = 1`: 1,112,357 times (5.56%)
- `length(A1) = 2`: 1,669,068 times (8.35%)
- `length(A1) = 3`: 2,216,588 times (11.08%)
- `length(A1) = 4`: 2,775,405 times (13.88%)
- `length(A1) = 5`: 3,333,920 times (16.67%)
- `length(A1) = 7`: 8,333,604 times (41.67%)

Expected value of `length(A1)`: 4.8603

Distribution of A's length after the second insertion (`A2`):

- `length(A2) = 0`: 689,171 times (3.45%)
- `length(A2) = 1`: 1,746,955 times (8.73%)
- `length(A2) = 2`: 2,511,990 times (12.56%)
- `length(A2) = 3`: 3,001,901 times (15.01%)
- `length(A2) = 4`: 3,202,618 times (16.01%)
- `length(A2) = 5`: 3,129,061 times (15.65%)
- `length(A2) = 6`: 3,434,722 times (17.17%)
- `length(A2) = 8`: 2,283,582 times (11.42%)

Expected value of `length(A2)`: 4.1555

> **Note**: We only perform **two** `simulate_insertion_tool` calls per trial because:
>
> - After the first insertion, B has 2 elements left.
> - After the second insertion, B has 1 element left.
> - The third (final) insertion would occur next, but since our focus is on estimating the *branching factor* of the decision tree, and because after the third insertion B becomes empty (terminating the process), we model the decision space using the first two steps to estimate average branching.

The number of insertion positions available at each step depends on the current length of A:

- First insertion: `length(A) + 1 = 6 + 1 = 7` possible positions.
- Second insertion: on average `E[length(A1)] + 1` positions.
- Third insertion: on average `E[length(A2)] + 1` positions.

Thus, the estimated **total number of decision paths** (i.e., sequences of insertion choices) is approximately:

$$((\text{length}(A) + 1) \cdot (E[\text{length}(A_1)] + 1) \cdot (E[\text{length}(A_2)] + 1)) \cdot P(13, 6) \cdot 13^3$$

Plugging in the values: (7 * 5.86 * 5.16) ≈ 211.5

Compare this to the static no-matching case, which had $\binom{9}{3} = 84$ final arrangements. The ratio is: 211.5/84 ≈ 2.5

This means the **number of decision paths is roughly 2.5 times larger** than the number of "final static combinations" under the no-matching assumption.

> **Crucially**, this estimate does **not** count final card combinations—it counts **decision sequences** (i.e., dynamic strategies). Because the simulation accounts for matching rules and the order-dependent nature of insertions (e.g., inserting B[0] then B[1] can yield a different outcome than the reverse), it captures the true **dynamic combinatorial complexity** of the game.

Therefore, the actual search space for optimal play—represented as a decision tree—is significantly larger than the static upper bound previously calculated. This highlights the immense scale of the problem and underscores the challenge in exhaustively searching for optimal strategies.

Exhaustive recursive search / brute-force recursion (Recursive_StrategyAndScore) attempts all possible options to guarantee finding the optimal solution, resulting in an enormous decision tree. However, the Verification Complexity Analysis yields a bound of $O(n^{3/2})$, indicating that the quality of any given strategy can be verified efficiently.

The problem is in NP and is likely NP-hard.

This resembles the Travelling Salesman Problem (TSP), where the decision to visit a particular city directly affects the distances and path choices for subsequent cities. Similarly, in our problem, a single "matching" operation fundamentally alters the "map" (i.e., list A), thereby influencing the scores (i.e., optimal card-playing decisions) of all subsequent "paths." This structural similarity strongly suggests that our problem is NP-hard. We formally prove this using a Turing reduction.

Proof of NP-hardness

Proof: The Card Game Optimization Problem (CARD_GAME_OPT) is NP-hard

In this proof, we follow standard theoretical computer science conventions. The input size of a problem instance is defined as the total number of bits required for its **standard binary representation**. Therefore, the input size of a numerical value N is O(log N). The time complexity of algorithms will be measured based on this input size.

1. Proof Strategy

To prove that the optimization problem `CARD_GAME_OPT` is NP-hard, we employ a **Turing Reduction** from a known NP-complete problem—the **Subset Sum Problem**.

The core logic of the proof follows a **proof by contradiction**: First, we assume that `CARD_GAME_OPT` is **not** NP-hard, meaning there exists a polynomial-time algorithm that can solve it. Then, we demonstrate how to use this hypothetical algorithm as an oracle to design **another** polynomial-time algorithm that solves the Subset Sum Problem.

If the above logic holds, it would mean that an NP-complete problem can be solved in polynomial time, leading to the conclusion that P=NP. This contradicts the fundamental assumption in computational complexity theory that P≠NP, thus our initial assumption must be false.

2. Problem Definitions

- **Subset Sum Problem**
  - **Input**: A **set** $S = \{s_1, s_2, \ldots, s_n\}$ consisting of n **distinct** positive integers, and a target integer T.
  - **Question**: Does there exist a subset $S' \subseteq S$ such that the sum of all elements in S' equals exactly T?
  - **Complexity**: This is a classic NP-complete problem[1]. We specifically refer to its formulation dealing with sets of distinct integers to ensure the validity of the reduction.
- **Card Game Optimization Problem (CARD_GAME_OPT)**
  - **Input**: Two card lists A and B.
  - **Question**: Find a card-playing strategy that **maximizes** the total score obtained, and return this maximum score.

3. Reduction Proof

Step 1: Contradiction Assumption

Assume that $CARD_GAME_OPT$ has a polynomial-time solution. We call this hypothetical algorithm $Solve_OPT$. We can view it as an oracle O that can compute $O(A, B) = max_score$ in polynomial time.

Step 2: Design a New Algorithm to Solve the Subset Sum Problem

We design a new algorithm called $SubsetSumSolver$ that takes an instance (S, T) of the Subset Sum Problem as input. This algorithm determines whether a solution exists by calling the oracle O.

A. Oracle Capability Encapsulation

The $SubsetSumSolver$ algorithm needs to "translate" the mathematical problem it wants to query into a game problem that the oracle O can understand. To this end, the algorithm internally implements a core function $MaxSubsetSum(S_sub)$.

- **Operation flow of function $MaxSubsetSum(S_sub)$:**
  1. The function receives S_sub as a parameter, where $S_sub \subset S$.
  2. Based on this parameter, the function programmatically constructs a game instance ☑ + S_sub). Here, X is a pre-selected integer sufficiently large to be greater than Sum(S).

> *Construction method: For any S, we can obtain X by calculating the sum of all elements in S and adding 1, i.e., X = Sum(S) + 1. ☑ and B=[X] + S_sub. This construction satisfies the rules of the card game and obviously holds for any S.

  3. The function passes the constructed instance I to the oracle O and obtains the return value max_score.
  4. The function computes and returns $max_score - 2X$.
- **Function Capability Analysis:** Since elements in S are distinct and X is extremely large, the only way to score is by matching two X cards. Therefore, since S_sub contains distinct elements, the result of

`max_score - 2X` is exactly the maximum subset sum of the input set `S_sub` (i.e., the sum of all elements in `S_sub`). Since constructing the instance, calling oracle `O`, and performing subtraction are all polynomial-time operations, the `MaxSubsetSum` function itself is also polynomial-time.

B. Main Body of `SubsetSumSolver` Algorithm

This algorithm uses its internal function `MaxSubsetSum` to determine whether a subset with sum `T` exists.

- **Algorithm execution flow `SubsetSumSolver(S, T)`:**

    1. **Initial check**: If `T < 0` or `T > MaxSubsetSum(S)`, then a solution obviously does not exist, and the algorithm returns "No".

    2. **Constructive solution**: a. Initialize target sum `T_rem = T`. b. Create a copy `S_rem` of `S` as the set of elements to be examined, sorted in descending order. c. Iterate through each element s_i in `S`: i. Remove s_i from `S_rem`. ii. Determine whether s_i should be added to the final solution: * If s_i `<= T_rem`, and `T_rem - `s_i `<= MaxSubsetSum(S_rem)` (if s_i is added to the solution set, it means the remaining elements can potentially sum to the remaining target)

        > This means if we select `$s_i$`, the remaining target sum `T_rem - $s_i$` **may possibly** be achieved by the remaining elements `S_rem`. The algorithm makes the decision to consider `$s_i$` as part of the solution and updates `T_rem = T_rem - $s_i$`.

    3. **Return result**: After iterating through all elements, if `T_rem` equals exactly `0`, it means the algorithm successfully found a combination of elements `S_solution` whose sum is `T`. The algorithm returns "Yes". Otherwise, it returns "No".

The total correctness of this algorithm includes:

```
A. Termination: The algorithm always halts in a finite number of steps.
B. Partial Correctness: The result after the algorithm halts is correct.
    To prove partial correctness, we need to prove:
        B1. Soundness: When the algorithm returns "Yes", the problem indeed has a
solution.
        B2. Completeness: When the problem has a solution, the algorithm will
definitely return "Yes".
```

Termination is obvious, as the algorithm terminates after traversing S.

Proof: Soundness

Question: If the `SubsetSumSolver` algorithm ends with T_rem = 0, why does this indicate that the algorithm successfully found a combination of elements `S_solution`?

1. Define Loop Invariant

Let us define an equation that always holds during the algorithm's loop process.

Let `T_rem(k)` be the value of `T_rem` after the algorithm has examined the first `k` elements. Let `S_solution(k)` be the `S_solution` set after the algorithm has examined the first `k` elements.

The **loop invariant** we want to prove is:

$$T_{\text{rem}}(k) + \sum_{s \in S_{\text{solution}}(k)} s = T$$

The behavior of T_rem is to be initialized to T, then continuously subtract potentially viable s_i, which is T minus the sum of elements already selected into the solution set.

This equation shows that after each step of the loop, **the total target T minus the sum of elements already selected** plus **the sum of elements already selected**, always equals **the original total target** `T`. The equation itself is obviously true.

2. Proof by Mathematical Induction

We will prove by mathematical induction that this invariant holds throughout the algorithm's execution.

- **Base Case (equivalent to the simplest case of a recurrence relation):** `k = 0` Before the loop begins (i.e., when 0 elements have been examined):
    - `S_solution(0)` is the empty set `[]`, so its element sum `Sum(S_solution(0)) = 0`.
    - `T_rem(0)` is initialized to `T`.
    - At this point, the invariant equation is: `T_rem(0) + Sum(S_solution(0)) = T + 0 = T`.
    - **Conclusion**: Before the loop begins, the invariant holds.
- **Inductive Step Assume** that after examining the `(k-1)`-th element, the invariant holds. That is:

    $$T_{\text{rem}}(k-1) + \sum_{s \in S_{\text{solution}}(k-1)} s = T$$

    Now, we analyze the two cases when the algorithm examines the `k`-th element s_k:

    - **Case 1: After adding s_k to the solution set `S_solution`** According to the algorithm's decision logic, this will happen. At this point, the state is updated as follows:
        - `S_solution(k)` = `S_solution(k-1) ∪ {`s_k`}`, therefore `Sum(S_solution(k)) = Sum(S_solution(k-1)) +` s_k.
        - `T_rem(k)` = `T_rem(k-1) -` s_k.

        We substitute the new state into the left side of the invariant equation: `T_rem(k) + Sum(S_solution(k)) = (T_rem(k-1) -` s_k`) + (Sum(S_solution(k-1)) +` s_k`) = T_rem(k-1) + Sum(S_solution(k-1))` According to our inductive hypothesis, the result of the above expression equals `T`. **Therefore, in this case, the invariant still holds after the `k`-th step.**
    - **Case 2: s_k is not added to the solution set `S_solution`** In this case, the state is updated as follows:
        - `S_solution(k)` = `S_solution(k-1)`, therefore `Sum(S_solution(k)) = Sum(S_solution(k-1))`.

- `T_rem(k)` = `T_rem(k-1)`.

  We substitute the new state into the left side of the invariant equation: `T_rem(k) + Sum(S_solution(k)) = T_rem(k-1) + Sum(S_solution(k-1))` According to our inductive hypothesis, the result of the above expression equals `T`. **Therefore, in this case, the invariant also still holds after the `k`-th step.**

- **Conclusion** Through mathematical induction, we have proven that this loop invariant `T_rem(k) + Sum(S_solution(k)) = T` always remains valid after each iteration of the algorithm.

3. Corollary

When the algorithm finishes traversing all `n` elements and terminates, this invariant still holds. Let `T_rem_final` and `S_solution_final` be the final state. We have:

$$T_{\text{rem_final}} + \sum_{s \in S_{\text{solution_final}}} s = T$$

At this point, the algorithm's return logic is to check whether `T_rem_final` equals `0`. If `T_rem_final = 0`, we can substitute it into the above proven equation:

$$0 + \sum_{s \in S_{\text{solution_final}}} s = T$$

$$\sum_{s \in S_{\text{solution_final}}} s = T$$

Proof complete. This shows that if and only if `T_rem` is 0 when the algorithm terminates, the sum of elements in our constructed solution set `S_solution` precisely equals the original target `T`.

Therefore, if T_rem becomes 0 when the algorithm ends, we can confirm that Sum(S_solution) must equal T.

Proof: Completeness

Question: If the problem truly has a solution, why must T_rem necessarily equal 0 when the algorithm ends (after the loop ends, can the invariant deduce the algorithm's correct result)?

Proposition to prove: For any Subset Sum Problem instance `(S, T)` that does have a solution, the `SubsetSumSolver` algorithm described in the report guarantees it will terminate and return "Yes" (i.e., final `T_rem = 0`).

Proof by contradiction:

1. **Make contradiction assumption** Assume the proposition to be proved is false. That is, **there exists** an optimal solution `S*` (whose sum of all elements is `T`), but the `SubsetSumSolver` algorithm **fails** after execution, ultimately returning "No" (i.e., `T_rem > 0` when the algorithm terminates).

2. **Analyze the inevitable cause of failure** If the algorithm's final result is incorrect, then during its execution of traversing elements in descending order, it must have made at least one "wrong" decision. We can pinpoint the moment when the algorithm makes its **first** mistake. This error must be:

   - The algorithm **did not select** an element that should belong to the optimal solution `S*`. We call this incorrectly "skipped" element `s*`.

Before considering `s*`, all elements larger than `s*` have been correctly processed.

3. **Analyze the conditions for the algorithm's wrong decision** According to the decision logic of `SubsetSumSolver`, the algorithm "skips" `s*` because it fails to satisfy the `if` condition. This condition is `s_i <= T_rem` and `T_rem - s_i <= MaxSubsetSum(S_rem)`.

   Analyze why this condition does not hold for `s*`:

   - **For `s* <= T_rem`**: When processing `s*`, the value of `T_rem` equals `T` minus the sum of all selected members of `S*` that are larger than `s*` (guaranteed by S in descending order).

   Therefore, the value of `T_rem` equals exactly `s*` plus the sum of all members of `S*` that are smaller than `s*`.

   That is:

   $$T_{\text{rem}} - s^* = \sum_{x \in S^, x < s^} x$$

   Therefore, `T_rem >= s*` must hold.

   - **Thus, the only reason for decision failure** can only be that the algorithm determined at that moment:

   $$T_{\text{rem}} - s^* > \text{MaxSubsetSum}(S_{\text{rem}})$$

4. **Derive contradiction** Let's analyze both sides of this inequality:

   - **Left side of inequality `T_rem - s*`**: According to the above analysis, this value equals **the sum of all elements in the optimal solution `S*` that are smaller than `s*`**.

   - **Right side of inequality `MaxSubsetSum(S_rem)`**: This value is the maximum subset sum that can be formed by the set of elements `S_rem` currently "to be examined" by the algorithm.

   - Those elements in the optimal solution `S*` that are smaller than `s*` must necessarily be a subset of the `S_rem` set. Therefore, the maximum subset sum that can be formed from `S_rem` must be greater than or equal to the sum of any of its subsets. Thus, logically it must hold that:

     $$\text{MaxSubsetSum}(S_{\text{rem}}) \geq \sum_{x \in S^, x < s^} x$$

     Using the equation obtained earlier, this is also:

     $$\text{MaxSubsetSum}(S_{\text{rem}}) \geq T_{\text{rem}} - s^*$$

     At this point, a contradiction appears:

   - Our **assumption** (algorithm failure) led to the conclusion: `T_rem - s* > MaxSubsetSum(S_rem)`.

   - The **logical deduction** yields the inevitable conclusion: `T_rem - s* <= MaxSubsetSum(S_rem)`.

   The same expression cannot simultaneously be both greater than and less than or equal to another expression. A mathematical contradiction arises.

5. **Draw conclusion** The source of the contradiction lies in our assumption in the first step. Therefore, the assumption that "a solution exists but the algorithm did not find it" is false.

   Thus the proposition to be proved is true: For any Subset Sum Problem instance with a solution, the `SubsetSumSolver` algorithm guarantees to find the solution and terminate with `T_rem = 0`.

This proof ensures that the decision logic of the `SubsetSumSolver` algorithm is sufficiently **complete** and will not miss any existing solution.

Therefore, the `SubsetSumSolver` algorithm has **total correctness**.

Step 3: Complexity Analysis and Contradiction Derivation

- **Complexity**: The `SubsetSumSolver` algorithm performs one traversal of the `n` elements in set `S`. In each loop iteration, it calls the polynomial-time `MaxSubsetSum` function once. Therefore, the running time of the entire `SubsetSumSolver` algorithm is polynomial.
- **Contradiction**: We have designed a polynomial-time algorithm (`SubsetSumSolver`) that can accurately solve the NP-complete "Subset Sum Problem". This directly leads to the conclusion that P=NP, contradicting the fundamental assumption in computational complexity theory that P≠NP.

Step 4: Conclusion

The source of the above contradiction lies in our assumption in the first step—that "`CARD_GAME_OPT` has a polynomial-time solution". Therefore, this assumption must be false.

Thus, the Card Game Optimization Problem `CARD_GAME_OPT` is NP-hard.

Conclusion 2: The generalized version of the card game score maximization problem is NP-hard.

Furthermore, according to previous conclusions, the steps of the strategy_TrueScore function are: Sort array B → Iterate through each element of B → Check if array A and the current B element match (have intersection) → Calculate the sum of the consecutive subsequence generated by the match

```
Size of array A is m
Size of array B is n
```

Sorting time complexity is O(n log n)

Linear scan to check if A and B[i] satisfy the matching rule has time complexity O(n)

Calculate the sum of the consecutive subsequence generated by the match (score from playing this card) has time complexity O(n)

Accumulate the scores corresponding to all elements of B to get the total score has time complexity O(n)

Total time complexity is O(n log n) + m*(O(n)+O(n)+O(n)) = O(n log n + mn)

This is polynomial time complexity, so we obtain:

Conclusion 3: The generalized version of the card game score maximization problem is in NP

Based on Conclusions 2 and 3:

The generalized version of the card game score maximization problem is NP-Complete